

---

# Documentación técnica del scraper de eventos culturales de Chile

Arquitectura, operación y mantenimiento

**Alcance:** extracción, normalización y sincronización del catálogo  
**Versión documental:** septiembre de 2026  
**Tecnologías principales:** Python, Playwright, NVIDIA NIM, SQLite y Supabase  
**Ejecución automatizada:** GitHub Actions

---

# Índice

|                                                      |          |
|------------------------------------------------------|----------|
| <b>1. Propósito y alcance</b>                        | <b>2</b> |
| 1.1. Principios de diseño . . . . .                  | 2        |
| <b>2. Arquitectura general</b>                       | <b>2</b> |
| 2.1. Patrones aplicados . . . . .                    | 2        |
| <b>3. Estructura del repositorio</b>                 | <b>3</b> |
| <b>4. Fuentes de datos</b>                           | <b>3</b> |
| <b>5. Flujo de procesamiento</b>                     | <b>4</b> |
| 5.1. Fase de extracción . . . . .                    | 4        |
| 5.2. Fase de procesamiento y publicación . . . . .   | 4        |
| <b>6. Modelo canónico de evento</b>                  | <b>5</b> |
| <b>7. Saneamiento, vigencia y filtro cultural</b>    | <b>5</b> |
| <b>8. IA, OCR y control de cuota</b>                 | <b>5</b> |
| <b>9. Estado local, checkpoints y recuperación</b>   | <b>6</b> |
| <b>10. Exportación y sincronización con Supabase</b> | <b>6</b> |
| <b>11. Configuración</b>                             | <b>6</b> |
| <b>12. Automatización con GitHub Actions</b>         | <b>7</b> |
| <b>13. Operación y mantenimiento</b>                 | <b>7</b> |
| 13.1. Comandos frecuentes . . . . .                  | 7        |
| 13.2. Cuando una fuente cambia o cae . . . . .       | 7        |
| <b>14. Seguridad, límites y consideraciones</b>      | <b>8</b> |
| <b>15. Documentos relacionados</b>                   | <b>8</b> |

## 1. Propósito y alcance

El proyecto construye y mantiene un catálogo de eventos culturales de Chile a partir de fuentes públicas: agendas municipales, espacios culturales, universidades, portales sectoriales y tiqueteras. Su objetivo no es copiar una página completa, sino convertir información pública heterogénea en registros comparables, trazables y vigentes.

El scraper abarca cuatro responsabilidades:

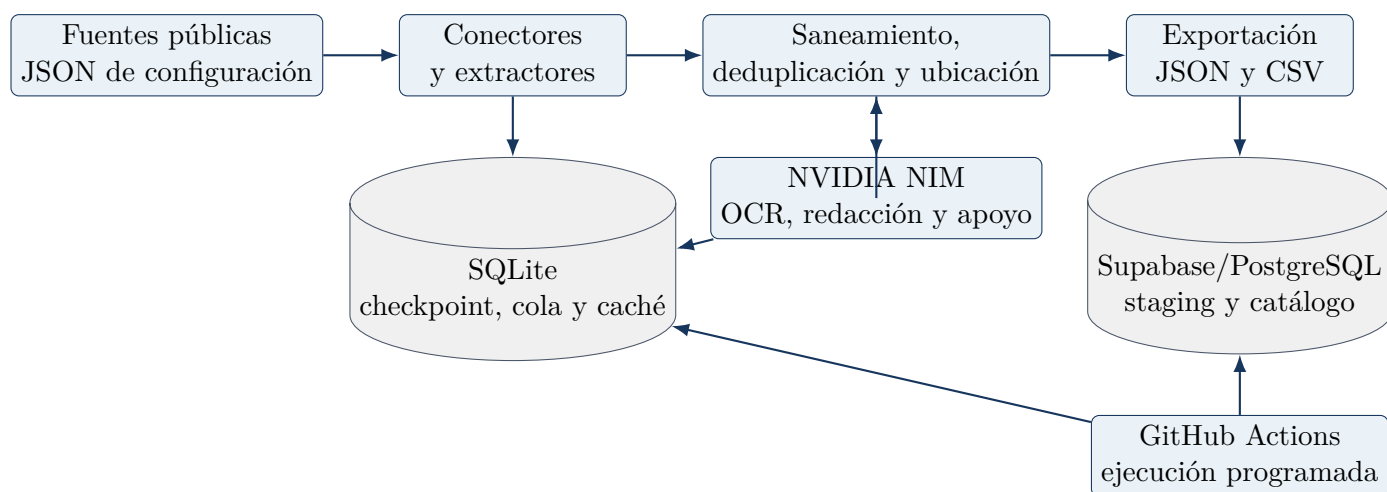
1. descubrir y extraer eventos desde fuentes configuradas;
2. normalizar fechas, ubicación, categorías, organizador y enlaces;
3. enriquecer solo los casos necesarios mediante OCR y redacción asistida por IA;
4. exportar un respaldo local y sincronizar el catálogo publicado con Supabase.

No forma parte de este documento el frontend, la autenticación de usuarios ni el algoritmo de recomendaciones. El scraper alimenta el catálogo que dichas capas consumirán posteriormente.

### 1.1. Principios de diseño

- **Trazabilidad:** cada evento conserva fuente, URL original, URL oficial cuando existe, método de extracción y momento de captura.
- **Preferencia por datos estructurados:** API, JSON-LD, WordPress REST y tarjetas declarativas se usan antes que OCR o modelos de IA.
- **Mínima presión sobre los sitios:** se respeta el ritmo por dominio, hay caché condicional HTTP y la concurrencia es acotada.
- **Idempotencia:** repetir una ejecución actualiza registros existentes y evita duplicados mediante identificadores estables y deduplicación.
- **Resiliencia:** una fuente caída, un afiche ilegible o una respuesta transitoria de NVIDIA no detienen el resto del proceso.
- **Vigencia:** se eliminan eventos vencidos y registros que una fuente completa ya no publica.

## 2. Arquitectura general



### 2.1. Patrones aplicados

| Patrón                    | Aplicación                                                                                                         |
|---------------------------|--------------------------------------------------------------------------------------------------------------------|
| Strategy                  | Cada conector implementa una estrategia de extracción independiente para una familia de sitios.                    |
| Factory/Registry          | <code>ConnectorFactory</code> selecciona el conector a partir del campo <code>connector</code> del JSON de fuente. |
| Repository                | <code>SourceRepository</code> carga, valida y entrega la configuración declarativa de <code>sources/</code> .      |
| Pipeline                  | <code>EventPipeline</code> orquesta fases en orden, sin acoplar la lógica a una página concreta.                   |
| Inyección de dependencias | HTTP, estado, exportador y servicios remotos pueden sustituirse en pruebas.                                        |

### 3. Estructura del repositorio

```

scrape_events.py Punto de entrada de la ejecución
eventos/ Paquete de extracción y procesamiento
  connectors/ Estrategias por tipo de fuente
  extractors/ Lectores reutilizables, por ejemplo JSON-LD
  services/ NVIDIA NIM y cliente REST de Supabase
  source_validation/ Pipeline independiente de validación de fuentes
sources/ Un JSON por fuente y plantillas reutilizables
state/ SQLite local: checkpoint, cola de IA y caché
data/ Exportaciones JSON, CSV, métricas y auditorías
supabase/migrations/ Migraciones SQL versionadas
tools/ Utilidades, entre ellas copia consistente de estado
tests/ Pruebas unitarias y de integración simulada
.github/workflows/ Automatización en GitHub Actions
docs/ Documentación del scraper

```

Los directorios `state/`, `data/`, cachés y el archivo `.env` son locales y no se publican en Git. Las fuentes, las migraciones, las pruebas y el workflow sí deben versionarse.

### 4. Fuentes de datos

La configuración vigente contiene 21 fuentes permanentes y una fuente estacional, activa únicamente en septiembre. Chile Cultura y Ticketplus aportan alcance nacional; las demás agregan profundidad territorial o temática.

| Fuente                           | Cobertura             | Método principal                      |
|----------------------------------|-----------------------|---------------------------------------|
| Chile Cultura                    | Nacional, 16 regiones | API pública paginada                  |
| Maipú en Común                   | Región Metropolitana  | API de aplicación mediante Playwright |
| Centro Cultural GAM              | Santiago              | JSON-LD                               |
| Centro Cultural La Moneda        | Santiago              | Tarjetas HTML declarativas            |
| Centro Cultural CEINA            | Santiago              | Tarjetas HTML declarativas            |
| Valpo Cultura                    | Valparaíso            | The Events Calendar API               |
| Espacio Cultural Viña del Mar    | Valparaíso            | The Events Calendar API               |
| Teatro Municipal de Viña del Mar | Valparaíso            | JSON-LD                               |
| Agenda Cultural UOH              | O'Higgins             | WordPress REST                        |
| Rancagua Cultura                 | O'Higgins             | JSON-LD                               |
| Agenda Universidad de Talca      | Maule                 | Tarjetas HTML declarativas            |

| Fuente                          | Cobertura             | Método principal                         |
|---------------------------------|-----------------------|------------------------------------------|
| Corporación Cultural de Iquique | Tarapacá              | Tarjetas institucionales                 |
| Centro de Arte Molino Machmar   | Los Lagos             | Tarjetas institucionales                 |
| Enoturismo Chile                | Interregional         | WordPress/EventON                        |
| FISA                            | Varias regiones       | Portafolio oficial                       |
| Espacio Riesco                  | Región Metropolitana  | Tarjetas HTML declarativas               |
| Parque Cultural de Valparaíso   | Valparaíso            | WordPress REST                           |
| Fundación CorpArtes             | Región Metropolitana  | Tarjetas HTML declarativas               |
| Cultura Vitacura                | Región Metropolitana  | Tarjetas HTML declarativas               |
| Santiago Cultura                | Santiago              | WordPress/EventON incremental            |
| Ticketplus Chile                | Nacional, 16 regiones | Selector regional y fichas estructuradas |
| FondasChile                     | Nacional, septiembre  | Tarjetas HTML declarativas               |

Cada archivo en `sources/` declara URL, región, comuna, organizador, conector, categorías por defecto, límites de paginación y, cuando corresponda, metadatos estáticos de ubicación. Añadir una fuente no exige modificar el archivo principal: primero se intenta configurar un conector existente; solo se crea código nuevo cuando la estructura no coincide con ninguno.

## 5. Flujo de procesamiento

### 5.1. Fase de extracción

1. Se cargan las fuentes habilitadas para el mes actual; FondasChile se omite fuera de septiembre.
2. Las fuentes se agrupan por dominio. Hasta cuatro dominios se procesan en paralelo, pero cada dominio conserva su propio ritmo.
3. Cada conector devuelve objetos `Event` con un esquema común.
4. Se aplican limpieza inicial, deduplicación interna y validación de disponibilidad de fichas.
5. Cada fuente exitosa o vacía de forma válida queda registrada en un checkpoint local.

La fase puede ejecutarse sin IA ni Supabase:

```
python scrape_events.py --phase extract --local-only
```

### 5.2. Fase de procesamiento y publicación

1. Se recupera la extracción compatible más reciente desde SQLite.
2. Se sanea el texto globalmente y se deduplican eventos entre fuentes.
3. Se aplican valores de ubicación declarados por fuente y reglas deterministas.
4. Si Supabase está habilitado, se detectan eventos nuevos, modificados, suprimidos o pertenecientes a organizadores bloqueados.
5. Solo el subconjunto pendiente pasa por OCR, redacción y enriquecimiento de ubicación.
6. Se ejecuta el filtro cultural, se descartan fechas inválidas o vencidas y se exportan JSON/CSV.
7. Se publica de forma atómica en Supabase y se reconcilian registros obsoletos de fuentes completas.

```
python scrape_events.py --phase process --supabase
```

La separación de fases permite conservar el trabajo de extracción si se agota una cuota de IA o falla temporalmente el destino remoto.

## 6. Modelo canónico de evento

Todo conector entrega el mismo modelo lógico. Los campos fundamentales son `title`, `start_date`, `end_date`, `venue`, `location`, `commune`, `region`, `organizer`, `categories`, `source_url` y `official_url`.

Las categorías son siempre una lista, no una cadena. Un mismo evento puede pertenecer simultáneamente a música, familiar y gratuito. En el JSON se guarda como arreglo; en CSV se serializa como JSON dentro de una celda; en Supabase se normaliza mediante `categories` y `event_categories`.

La ubicación se resuelve de forma transparente y cada evento incluye:

- `location`: texto canónico que nunca queda vacío;
- `location_precision`: dirección, recinto, comuna, región o modalidad en línea;
- `location_source`: dato publicado, configuración de fuente, regla determinista, IA o fallback geográfico.

No se inventan direcciones. Cuando la evidencia solo permite identificar una comuna o región, se conserva exactamente ese nivel y se marca la menor precisión.

## 7. Saneamiento, vigencia y filtro cultural

El saneador común elimina HTML residual, reglas CSS, scripts, shortcodes de WordPress/WPBakery y descripciones desproporcionadas. Esto evita que cambios visuales de un sitio se conviertan en contenido del catálogo.

Un evento se conserva solo si tiene fecha de inicio válida y todavía está vigente. La validación de URL retira fichas confirmadas con HTTP 404/410 o páginas de error inequívocas; ante timeout, bloqueo temporal o error 5xx el registro se conserva para no borrar información por una falla pasajera.

El filtro cultural funciona después de sanear y consolidar, antes de exportar. Tiene tres resultados: *aceptado*, *revisión* y *excluido*. Se incluye una actividad cuando su finalidad principal o una dimensión sustantiva permite crear, expresar, conservar, interpretar o compartir cultura. El formato por sí solo —por ejemplo “feria” o “exposición”— no determina el resultado. La auditoría queda en `data/cultural_filter_audit.json`.

## 8. IA, OCR y control de cuota

NVIDIA NIM se usa como respaldo, no como requisito de extracción. Las fuentes con API, JSON-LD, WordPress REST o tarjetas declarativas se procesan sin consumir IA. La IA interviene únicamente en estos casos:

- OCR de afiches cuando existe imagen y no hay texto suficiente;
- redacción editorial de texto público, preservando el texto de origen para trazabilidad;
- resolución por lotes de ubicaciones incompletas.

El límite global se configura en 40 solicitudes por minuto. Todas las llamadas, incluso las iniciadas por fuentes concurrentes, pasan por el mismo regulador. Las respuestas 429, **Retry-After** y secuencias de timeouts activan pausas controladas; los pendientes no se pierden, sino que pasan a una cola SQLite para una ejecución futura.

Antes de usar IA, Supabase compara URL original y una clave combinada de título, fecha y lugar. Si el evento ya existe y no cambió, se reutilizan OCR y texto editorial previamente almacenados.

Esta deduplicación reduce costo, tiempo y consumo de cuota.

## 9. Estado local, checkpoints y recuperación

El archivo `state/pipeline.sqlite3` conserva datos operativos, no el catálogo público. Incluye checkpoints por fuente, una extracción consolidada, cola de enriquecimiento, caché de resultados de IA, métricas y caché HTTP condicional.

En GitHub Actions, el estado se guarda como artefacto después de la extracción y nuevamente al finalizar. Una ejecución siguiente restaura el último artefacto disponible. Por ello, si falla la fase de enriquecimiento, no se requiere volver a recorrer todas las páginas: la siguiente corrida retoma los eventos y pendientes conservados.

No deben ejecutarse simultáneamente dos procesos que compartan el mismo estado local o las mismas credenciales de IA. El workflow usa un grupo de concurrencia para impedir que dos ejecuciones programadas se solapen.

## 10. Exportación y sincronización con Supabase

El respaldo local se escribe en `data/events.json`, `data/events.csv`, archivos separados por región, `data/source_metrics.json` y `data/run_summary.json`. Al final de cada corrida se informa un total consolidado y duración completa.

La sincronización remota usa la API REST de Supabase con una clave secreta exclusiva del proceso de ingesta. El flujo remoto es:

1. transformar eventos en entidades normalizadas: fuentes, organizadores, comunas, categorías, eventos, procedencia y medios;
2. cargar fragmentos en la tabla privada `catalog_staging`;
3. ejecutar una RPC transaccional que publica todo el lote o no publica nada;
4. eliminar eventos vencidos y filas no vistas por fuentes cuya extracción terminó correctamente.

Las consultas y cargas REST idempotentes reintentan fallas transitorias de red, timeout o disponibilidad temporal. La RPC de publicación no se repite automáticamente si su respuesta se pierde, porque podría haber finalizado y eliminado el staging; así se evita una publicación ambigua o duplicada.

Los organizadores bloqueados y los eventos suprimidos individualmente se excluyen antes de OCR, redacción y publicación. De este modo, una solicitud de retiro no se revierte en la siguiente ejecución.

## 11. Configuración

La configuración sensible vive en `.env` y nunca debe subirse al repositorio. Las variables mínimas para sincronizar son:

```
SUPABASE_ENABLED=true
SUPABASE_URL=https://TU_PROYECTO.supabase.co
SUPABASE_SECRET_KEY=sb_secret...
NVIDIA_API_KEY=...
```

Los parámetros operativos más relevantes son:

| Variable                        | Finalidad                                                             |
|---------------------------------|-----------------------------------------------------------------------|
| SOURCE_WORKERS                  | Dominios procesados en paralelo; valor recomendado: 4.                |
| HTTP_CACHE_ENABLED              | Activa revalidación condicional mediante ETag y Last-Modified.        |
| NVIDIA_RPM_LIMIT                | Límite global de solicitudes a NVIDIA; configurado en 40.             |
| NVIDIA_OCR_MAX_IMAGES           | Máximo de afiches OCR por ejecución.                                  |
| NVIDIA_REWRITE_ENABLED          | Activa o desactiva la redacción editorial.                            |
| SUPABASE_DELETE_EXPIRED         | Elimina eventos ya terminados al sincronizar.                         |
| SUPABASE_RECONCILE_STALE        | Retira eventos no vistos por fuentes completas.                       |
| SUPABASE_MAX_RETRIES            | Reintentos de operaciones REST idempotentes ante fallas transitorias. |
| SUPABASE_RETRY_MAX_WAIT_SECONDS | Espera máxima entre reintentos REST.                                  |

## 12. Automatización con GitHub Actions

El workflow instala dependencias, Chromium para Playwright, ejecuta las pruebas, restaura checkpoints, extrae, conserva un artefacto, enriquece y sincroniza. Las credenciales se almacenan como *Repository secrets*: SUPABASE\_URL, SUPABASE\_SECRET\_KEY y NVIDIA\_API\_KEY.

La programación vigente es diaria a las 00:15 en la zona America/Santiago. El workflow conserva también `workflow_dispatch`, por lo que puede iniciarse manualmente desde la pestaña Actions. La zona horaria evita tener que ajustar el cron por horario de verano.

## 13. Operación y mantenimiento

### 13.1. Comandos frecuentes

```
# Instalaci\'}{o}n en Windows
instalar.bat

# Ejecuci\'}{o}n completa local
ejecutar.bat

# Pruebas
python -m unittest discover -s tests -v

# Solo extracci\'}{o}n, sin IA ni Supabase
python scrape_events.py --phase extract --local-only

# Procesar un checkpoint y sincronizar
python scrape_events.py --phase process --supabase

# Aplicar migraciones de Supabase en Windows
migrar_supabase.bat
```

### 13.2. Cuando una fuente cambia o cae

1. revisar el resumen de fuente y confirmar si falló, quedó parcial o está legítimamente vacía;
2. comprobar la página pública y sus términos de uso antes de cambiar la extracción;

3. preferir API, JSON-LD o endpoint público sobre selectores visuales frágiles;
4. actualizar solo el JSON de fuente cuando un conector declarativo sea suficiente;
5. agregar o corregir pruebas con una muestra representativa antes de habilitar la fuente;
6. si la fuente falla, mantenerla fuera de la reconciliación: el pipeline no borra su catálogo por una extracción parcial o fallida.

La guía operativa detallada está en `sources/GUIA_AGREGAR_Y_MANTENER_FUENTES.md`.

## 14. Seguridad, límites y consideraciones

Se consultan únicamente recursos públicos. Antes de añadir una fuente se deben revisar términos de uso, `robots.txt`, condiciones de reutilización y frecuencia razonable de consulta. El sistema no debe intentar evadir bloqueos ni acceder a contenido privado.

Las claves secretas de Supabase se usan solamente por el scraper o GitHub Actions; nunca deben llegar al frontend, a archivos versionados ni a logs. El OCR y la redacción automatizada no sustituyen una revisión editorial ni garantizan, por sí solos, conformidad con políticas de terceros.

## 15. Documentos relacionados

- `README.md`: instalación, comandos cotidianos y visión rápida.
- `docs/ARCHITECTURE.md`: detalle de componentes y patrones.
- `docs/SOURCES.md`: procedencia y tratamiento de cada fuente.
- `docs/RESUMABLE_PIPELINE.md`: checkpoints, cola de IA y recuperación.
- `docs/SUPABASE.md`: migraciones, seguridad y operación de la sincronización.
- `docs/DEFINICION_Y_FILTRO_CULTURAL.md`: criterio de inclusión cultural.